



Review on Meta-Learning in Neural Networks

By Aadya Arun Talreja, Amogha V Prasad, & Freya D Raja

(1RVU23CSE003)

(1RVU23CSE046)

(1RVU23CSE158)



Week 1 Agenda: Meta Learning

- Introduction to Meta Learning
- Background of Meta Learning
- Related Fields
- Taxonomies
- Meta Learning Landscapes



Introduction to Meta-Learning

What is Meta-Learning?

- Model gains experience over multiple learning episodes — “*learning to learn*.”
- A learner’s performance at solving tasks drawn from a given task family **improves as more tasks are seen**.
- Focuses on **generalization** across tasks rather than performance on a single task.

Why It Matters:

- Improves **data efficiency**, **knowledge transfer**, and enables **unsupervised** or **self-directed learning**.
- Particularly important for settings with limited labeled data or the need for fast adaptation.

Applications

- Few-shot image recognition
- Unsupervised and self-directed reinforcement learning
- Hyperparameter optimization and neural architecture search
- Algorithm selection based on dataset feature



Background of Meta-Learning

Why Meta-Learning?

- Traditional ML models require **large labeled datasets** and must be **retrained** for each new task
- Humans learn efficiently — we generalize across experiences.
- **Meta-learning aims to bring this adaptability to machines.**

Core Framework - Two Stage Process

- **Base Learning (Inner Loop):** Learns a model for a specific task using a dataset and objective (e.g., loss function).
- **Meta-Learning (Outer Loop):** Learns how to improve the base learner — optimizing generalization, speed, or adaptability. Uses performance across tasks to improve learning strategy

Task Distribution Setup

- Tasks are sampled from a distribution: $T \sim p(T)$
- Each task includes a **Training Set (Support Set)** and **Validation Set (Query Set)**
- **Goal:** Leverage prior tasks to quickly learn new ones.

Different Types of Meta Learning Views

1. Task-Distribution View

- Learn a meta-learner that performs well on tasks from a distribution $p(\mathcal{T})$.
- **Meta-Knowledge ω** : Captures *how to learn* across tasks. (*'How to Learn'* $\rightarrow$ *Across-task Knowledge*)

Phases:

- **Meta-Training**: Learn from training (support) + validation (query) splits of tasks from $p(\mathcal{T})$.
- **Meta-Testing**: Apply ω to unseen tasks.

Meta-overfitting: When ω doesn't generalize well to new tasks.

$$\min_{\omega} \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} \mathcal{L}(\mathcal{D}; \omega) \quad (2)$$

$$\omega^* = \arg \max_{\omega} \log p(\omega | \mathcal{D}_{source}) \quad (3)$$

$$\theta^{*(i)} = \arg \max_{\theta} \log p(\theta | \omega^*, \mathcal{D}_{target}^{train(i)}) \quad (4)$$

Different Types of Meta Learning Views

2. Bilevel Optimization View

- Formulated as a **hierarchical optimization problem**:
 - **Inner loop**: Learns task-specific model parameters θ
 - **Outer loop**: Learns meta-parameters ω
- Outer loop = *Leader* (sets ω)
- Inner loop = *Follower* (it optimizes θ based on ω , but can't modify ω)

Even a single task ($M=Q=1$) can be used by generating multiple episodes via data splits.

$$\overset{\text{meta-learner}}{\omega^*} = \arg \min_{\omega} \sum_{i=1}^M \overset{\text{outer objective}}{\mathcal{L}^{\text{meta}}(\theta^{*(i)}(\omega), \omega, \mathcal{D}_{\text{source}}^{\text{val}(i)})} \quad (5)$$

$$\text{s.t. } \overset{\text{base learner/train}}{\theta^{*(i)}(\omega)} = \arg \min_{\theta} \overset{\text{inner objective}}{\mathcal{L}^{\text{task}}(\theta, \omega, \mathcal{D}_{\text{source}}^{\text{train}(i)})} \quad (6)$$

Different Types of Meta Learning Views

3. Feed-Forward Model View (Amortized Meta-Learning)

- Learns a function $g_{\omega}(\mathcal{D}_{\text{train}})$ that maps training data directly to a model.
- **No inner-loop optimization during test time.**
- Fast adaptation = forward pass only.

Amortization: Training "pre-computes" the learning cost, so testing is lightweight. The learning cost is "paid" during meta-training.

Amortized = One-time effort during training that reduces future cost.

Fast: no inner-loop learning is needed during testing

→ During meta-training, g_{ω} learns to approximate the *result* of what inner-loop optimization would have produced.

→ At meta-test time, you skip the optimization and just apply g_{ω} once.

$$\min_{\omega} \mathbb{E}_{\substack{\mathcal{T} \sim p(\mathcal{T}) \\ (\mathcal{D}^{tr}, \mathcal{D}^{val}) \in \mathcal{T}}} \sum_{(\mathbf{x}, y) \in \mathcal{D}^{val}} \left[(\mathbf{x}^T g_{\omega}(\mathcal{D}^{tr}) - y)^2 \right] \quad (7)$$



Historical Context of Meta-Learning

Early Foundations:

- 1987: Jürgen Schmidhuber introduces *self-referential learning* using evolutionary methods.
- 1990s–2000s:
 - ◆ Thrun et al. define *learning to learn*.
 - ◆ Bengio et al. explore biologically plausible learning rules.
 - ◆ Early meta-learning via **gradient descent** proposed in 1991.

Expansion:

- 1995: Applied in reinforcement learning.
- 2000s: Growth with better compute, datasets, and deep learning tools.

Modern Breakthroughs:

- MAML (2017): General-purpose gradient-based meta-learner.
- Prototypical & Matching Networks (2016): Few-shot classification.
- Growth into Meta-RL, NAS, and Hyperparameter Optimization.



Model-Agnostic Meta-Learning (MAML) Algorithm

MAML is a meta-learning algorithm that learns a model initialization for fast adaptation. It enables learning new tasks using only a few gradient updates and limited data. It fits the meta-learning paradigm by training across many tasks to learn “how to learn”. Instead of learning final task solutions, it learns an adaptable starting point for new tasks. MAML is ideal for few-shot learning because it’s trained to adapt with very little data.

- During training, it simulates few-shot conditions using small support and query sets.
- Works across domains like classification, regression, and reinforcement learning.
- Adapts quickly to new tasks without retraining from scratch.
- Requires computing gradients through gradients, which can be memory intensive.
- Performance depends on task similarity and careful tuning of learning rates .



Concept	Core Goal	Meta-Learning?	Notes
Transfer Learning (TL)	Transfer knowledge from a source task to a target task	NO	No meta-objective; direct transfer of parameters or features
Domain Adaptation (DA)	Adapt to target domain with different input distribution	Typically	Adaptation uses target data but not optimized via meta-objective
Domain Generalization (DG)	Generalize to unseen domains	Typically	Meta-learning can improve generalization, but DG itself lacks meta-objective
Continual Learning (CL)	Learn sequentially without forgetting	Mostly	Shares goal of fast learning but usually lacks explicit meta-objective
Multi-Task Learning (MTL)	Solve multiple tasks together	Typically	Can benefit from meta-learning to prioritize or relate tasks
Hyperparameter Optimization (HO)	Tune learning settings	YES:if gradient-based NO:if black-box (e.g., Bayesian/Random search)	End-to-end HO aligns with meta-learning; black-box does not
Hierarchical Bayesian Models (HBMs)	Learn priors for grouped models	Conceptually similar	Probabilistic view of meta-learning; useful theoretical analogy
AutoML	Automate ML pipeline	Partially	Meta-learning is one specialized part of AutoML



Previous Taxonomy

Optimization - These treat meta-learning as a **bi-level optimization problem**, with a **clear inner loop** and **outer loop**. (MAML is an example of optimization)

Strengths: Highly generalizable to unseen tasks, Strong theoretical foundation.

Limitations: Computationally expensive, Memory intensive for long inner-loop

Black Box / Model-based - These embed the entire task in a single feed-forward model, usually a neural network, with no explicit optimization during test time.

Strengths: Fast inference (just a forward pass), Easier to train (no nested optimization).

Limitations: Weaker generalization to out-of-distribution tasks, Poor scalability to large support sets (limited memory capacity), Asymptotically weaker performance compared to optimization-based methods.

Metric-Learning - These methods don't train per-task models. Instead, they compare new data points directly to training examples using learned embeddings. Think of "**learning to compare**" rather than "learning to classify."

Strengths: Extremely data-efficient (especially for few-shot classification), Simple and fast.

Limitations: Often limited to classification tasks, May not generalize well beyond few-shot settings, Performance can degrade if tasks



Proposed Taxonomy

Meta-Representation (“What?”) *What kind of knowledge is being learned during meta-learning?*

This refers to what the meta-learning system is trying to acquire across tasks. Examples include: Initial model parameters, Optimization rules or update functions, Neural activations or embeddings, Programs or symbolic representations.

This axis answers: What form does the transferable meta-knowledge take?

Meta-Optimizer (“How?”) *How is the meta-learning process performed? (i.e., what optimization method is used?)*

This axis refers to the mechanism used to optimize the meta-knowledge (typically the outer loop in meta-learning): Gradient-based optimization, Reinforcement Learning (RL).

This axis answers: How is the meta-learner itself trained?

Meta-Objective (“Why?”) *What is the goal or purpose of meta-learning?*

This is determined by: The choice of outer loss function L_{meta} , The distribution of tasks $\mathbf{p}(\mathbf{T})$, The data flow and structure of episodes.

This axis answers: Why are we doing meta-learning? What property or behavior are we optimizing for?

Overview of Meta Learning Landscape

META LEARNING

META-OPTIMIZER

GRADIENT

REINFORCEMENT
LEARNING

EVOLUTION

META REPRESENTATION

PARAMETER
INITIALIZATION

OPTIMIZER

BLACK-BOX MODEL

EMBEDDING

MODULES

INSTANCE
WEIGHTS

ATTENTION

HYPERPARAMETERS

ARCHITECTURE

NOISE
GENERATOR

CURRICULUM

DATASET/
ENVIRONMENT

LOSS/
REWARD

EXPLORATION
POLICY

DATA
AUGMENTATION

META-OBJECTIVE

MANY/FEW-SHOT

MULTI/SINGLE TASK

ONLINE/OFFLINE

NET/ASYMPTOTIC
PERFORMANCE



Meta-Representation

Definition: The *form* of knowledge that is meta-learned and transferred across tasks.

What Can Be Meta-Represented?

→ Initial Model Parameters

- ◆ E.g., **MAML** learns a good initialization for fast task adaptation.
- ◆ Extensions: learn only a **subset** (e.g., per-layer, subspace, scale/shift) to reduce complexity.
- ◆ Challenge: a single initialization might not fit all tasks → use **mixture models** over initializations.

→ Optimizers and Update Rules

- ◆ Learn *how to update parameters* instead of just initializing them.
- ◆ Can include: Step sizes, Preconditioning matrices, Fully learned optimizers (e.g., RNN-based)

→ Loss Functions

- ◆ Learn custom loss functions tailored to improve generalization or robustness.
- ◆ Can aid in dealing with noise, domain shifts, or optimization of non-differentiable objectives.

→ Embeddings and Metrics

- ◆ Learn embedding functions for **metric-based learning** (e.g., prototypical networks).
- ◆ Represent tasks in a latent space suitable for comparison and classification.

→ Architectures and Modules

- ◆ Learn which parts of a network to share or synthesize per task.
- ◆ Includes **hypernetworks**, **modular meta-learning**, and **differentiable architecture search (DARTS)**.

→ Datasets and Labels (Transductive Representations)

- ◆ Learn synthetic support sets, labels, or data selection strategies (e.g., dataset distillation).



Meta-Optimizer

Definition: The algorithm or method used to optimize meta-knowledge during meta-training.

Types of Meta-Optimizers

→ Gradient-Based Optimization

- ◆ Most widely used (e.g., SGD, Adam).
- ◆ Used in methods like MAML and Reptile.
- ◆ May involve second-order gradients (compute intensive).

→ Learned Optimizers

- ◆ Optimizers themselves are neural networks (e.g., RNNs predicting updates).
- ◆ Examples: coordinate-wise optimizers, learned step size functions, preconditioners.

→ Zeroth-Order Methods

- ◆ Use only function evaluations (e.g., loss), not gradients.
- ◆ Useful when gradients are unavailable or expensive.
- ◆ Compete with Bayesian Optimization methods.

→ Reinforcement Learning (RL)

- ◆ Meta-learner is trained to maximize rewards (e.g., learning performance or speed).
- ◆ Used for discrete choices like data augmentation, module selection, or symbolic loss design.

→ Evolutionary Algorithms

- ◆ Use population-based search to evolve meta-parameters or architectures.
- ◆ Particularly useful in neural architecture search (NAS).

→ Hybrid Optimizers

- ◆ Jointly learn initializations + optimizers (e.g., MAML + learned update rule).
- ◆ Combine gradient-based methods with evolutionary or RL methods for complex domains.



Meta-Objective

Definition: The *goal* of the meta-learning process, determining **what performance criteria** the meta-learner optimizes.

Forms of Meta Objectives

→ Few-Shot Learning

- ◆ Most common meta-objective: perform well with few examples.
- ◆ Evaluate validation accuracy after few updates using support set.

→ Fast Many-Shot Optimization

- ◆ Improve convergence speed on large tasks.
- ◆ Meta-learn to reduce training time or number of updates needed.

→ Robustness Objectives

- ◆ Domain Shift Robustness: Generalize across input distributions.
- ◆ Label Noise Robustness: Handle incorrect or noisy labels.
- ◆ Adversarial Robustness: Withstand perturbations during inference.

→ Learning Efficiency

- ◆ Learn faster optimizers, curriculum strategies, or better sample weighting.
- ◆ Meta-learn strategies for better generalization per unit of compute or data.

→ Self-Supervised and Semi-Supervised Learning

- ◆ Meta-objectives defined over unlabeled data or auxiliary tasks.
- ◆ Meta-learn to generate useful unsupervised tasks or labels.

→ How It's Used in Practice

- ◆ Validation loss after task-specific training acts as the meta-objective.
- ◆ Determines how useful the current meta-knowledge is for new tasks.
- ◆ Can be task-specific, distribution-agnostic, or goal-driven (e.g., maximize reward, minimize data use).

Applications of Meta-Learning

1. Few-Shot Learning

- Learn to classify or predict from just **a few examples**.
- Used in:
 - ◆ **Image classification** (e.g., Omniglot, minImageNet)
 - ◆ **Speech and NLP** (low-resource languages)

2. Reinforcement Learning (Meta-RL)

- Agents learn how to **adapt policies** across tasks/environments.
- Enables **fast policy updates** in new environments.
- Used in:
 - ◆ Robotics
 - ◆ Game playing

3. Neural Architecture Search (NAS)

- Automates the design of **deep learning architectures**.
- Meta-learns the **best-performing model structure**.
- Used in:
 - ◆ Image recognition
 - ◆ NLP pipelines

4. Hyperparameter Optimization (HO)

- Meta-learns effective settings for model training.
- Replaces manual/grid search with intelligent tuning
- Used in:
 - ◆ Learning rate optimization
 - ◆ Regularization parameter tuning

5. AutoML and Algorithm Selection

- Meta-learns **which ML algorithms work best** on different datasets.
- Used in:
 - ◆ Automated data science pipelines
 - ◆ Model selection in production ML systems

6. Unsupervised & Self-Supervised Learning

- Meta-learns **useful representations or pretext tasks** from unlabeled data.
- Used in:
 - ◆ Label-efficient learning
 - ◆ Better transfer to downstream tasks

Inference of Overview of Meta-Learning

Meta-learning = “Learning to Learn”

Enables models to generalize across tasks and adapt quickly with minimal data.

Three Core Views:

- **Task Distribution View** – Learn across a range of tasks
- **Bilevel Optimization View** – Inner vs. outer loop learning
- **Feed-Forward View** – Direct task-to-model mapping (amortized)

Three-Axis Taxonomy:

- **Meta-Representation** (What is learned)
- **Meta-Optimizer** (How it's learned)
- **Meta-Objective** (Why it's learned)

Why It Matters

- Makes machine learning more efficient, flexible, and human-like.
- Powers modern applications in few-shot learning, reinforcement learning, AutoML, and more.

Future Outlook

- Broader generalization to unseen domains
- Improved scalability and interpretability
- Integration with symbolic reasoning and continual learning



Week 2 Agenda: MAML Implementation & Encoding Techniques

→ Implementation of MAML Algorithm



MAML-GAN for Few-Shot Distribution Learning

Objective:

- Train a **meta-generator** that can adapt to new distributions (e.g., Gaussians with unknown mean/std) using few inner updates.

Architecture Components:

Generator:

- Input: Noise vector z + conditioning [mean, std]
- Output: 1D generated sample

Architecture Components:

Discriminator:

- Input: Data sample x + same conditioning vector
- Output: Scalar real/fake score

Task Sampling

- Each task is a Gaussian distribution
- Parameters (mean, std) sampled randomly per task
- Conditioning vector = [mean, std] for generator and discriminator

Inner and Outer Loop Training (MAML-GAN)

Inner Loop (Per Task):

- Clone generator & discriminator
- Train on one Gaussian distribution
- **WGAN losses** used for stable GAN training
 - **Discriminator loss:**
$$L_D = - (E[D(x_{\text{real}})] - E[D(x_{\text{fake}})])$$
 - **Generator loss:**
$$L_G = -E[D(x_{\text{fake}})]$$

Outer Loop (Meta-Update):

- Aggregate post-adaptation losses across tasks
- Backpropagate to meta-generator and meta-discriminator
- Update shared initialization using Adam optimizer

Regularization:

- WGAN weight clipping applied for stability
- Clip all discriminator weights within ± 0.1



Generated Results & Key Observations

Training Output:

- Every 100 iterations: Logs average generator & discriminator loss
- Every 500 iterations: Displays few samples from meta-generator

After Training (Test-Time Adaptation):

- Evaluate on a new **unseen Gaussian distribution**
- Compare real vs. generated histograms

Insights:

- Generator learns a **prior that quickly adapts** to new task conditions
- Generated distribution closely matches true task distribution
- Validates the **few-shot generative ability** enabled by meta-learning

Applications:

- Data augmentation
- Personalized generative models
- Rapid adaptation in low-data environments



Week 3 Agenda: Meta Learning Meets Generative Models

- Comparative study of MAML, Reptile and other algorithms
- How GANs, VAEs, Diffusion complement meta-learning



Comparative study of MAML, Reptile and other algorithms

Algorithm	Gradient Type	Key Feature	Strength	Limitation
MAML	Second-order	Learns init that adapts well	Accurate meta-gradients	Slow, complex
Reptile	First-order	Pulls weights toward task-specific	Fast, easy to implement	Slightly less optimal updates
FOMAML	First-order approx	No second-order terms	Faster MAML alternative	Less precise than MAML
Meta-SGD	Second-order	Learns init and learning rates	Flexible learning dynamics	Higher parameter count



How GANs, VAEs, Diffusion complement Meta-Learning

Generative models struggle in low-data regimes due to overfitting, unstable training, or slow convergence. Meta-learning addresses this by training models to quickly adapt to new tasks using only a few examples.

GANs require large datasets and are prone to mode collapse when data is limited. Meta-learning helps by learning good initial weights for the generator and discriminator, enabling GANs to adapt quickly to new domains (e.g., MetaGAN trained on digits 0–8 can generate digit 9 using just 10 samples).

VAEs perform better in low-data settings due to their regularized latent space. However, their encoders and decoders may still underfit unfamiliar domains. Meta-learning improves this by allowing fast adaptation in the latent space, making them ideal for few-shot generative tasks like MetaVAE.

Diffusion models are highly stable and robust but slow to adapt. In few-shot scenarios, meta-learning helps diffusion models learn task-specific denoising processes efficiently, enabling faster generation with high quality.

In summary, meta-learning enhances each generative model by enabling **faster adaptation**, **better generalization**, and **greater sample efficiency** in low-data environments.

Meta-GAN with MAML & Reptile: Rapid Few-Shot Image Generation (ICLR 2022)

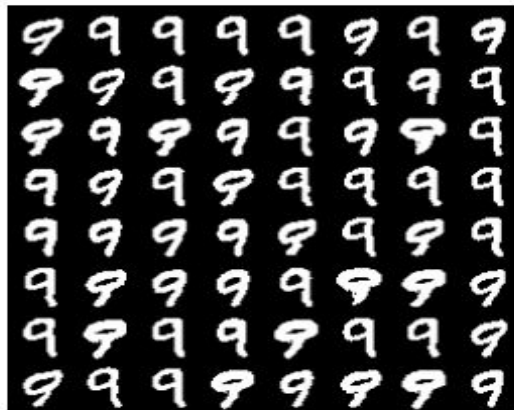
This work adapts MAML and Reptile to GANs, enabling them to generate realistic images in unseen domains using only a few examples. Meta-training on digits '0'-'8' allowed the model to adapt to digit '9' using just 10 images. Both CNN (Res-SNGAN) and Transformer (TransGAN) GANs were tested. Meta-trained models achieved similar quality to baseline models trained on thousands of samples, with 3–5× faster convergence. Reptile offered better training efficiency, showing the promise of meta-learning in few-shot generative modeling.



(a) Baseline Training



(b) MAML Meta-Training



(c) Reptile Meta-Training



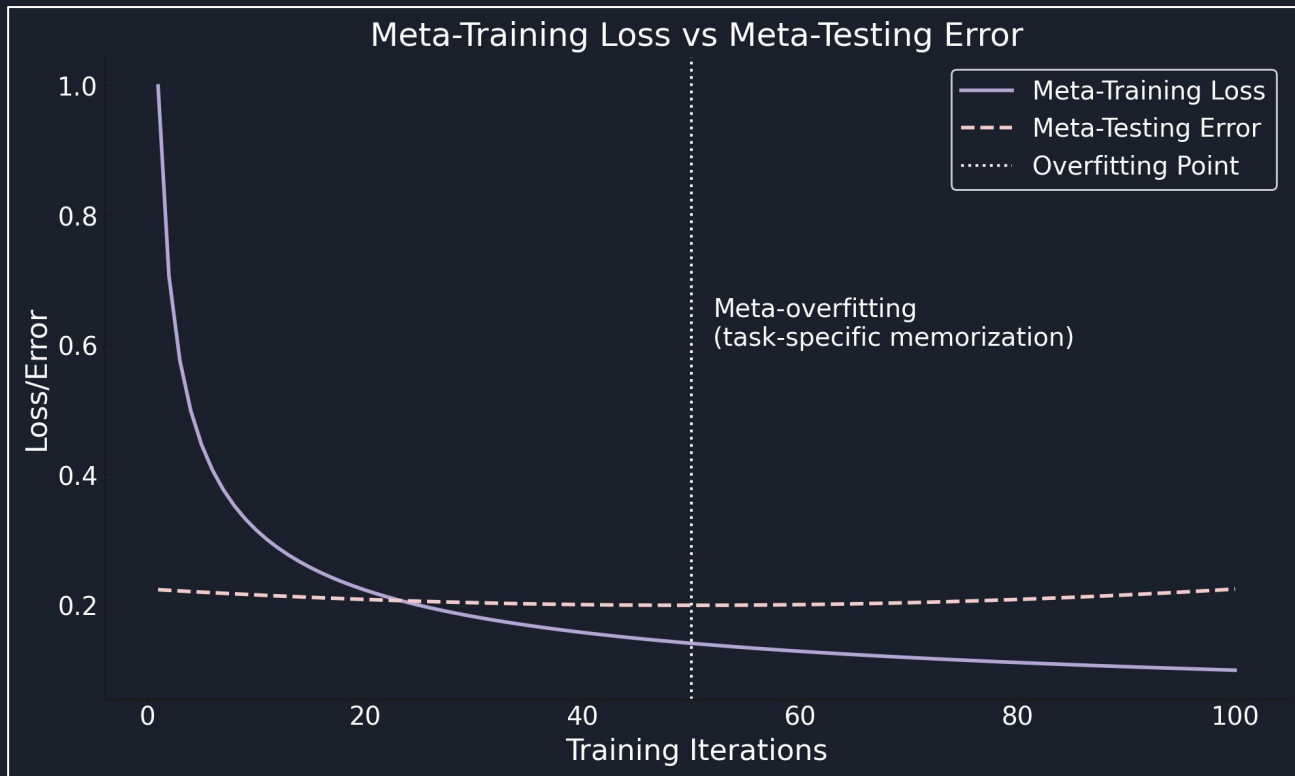
Week 4 Agenda: Meta Learning Theory: Generalization & Optimization

- Theoretical Foundations of generalization
- Explore Meta over-fitting and meta-regularization
- Gradient-based Meta Learning theory
- Meta Learning vs. Transfer Learning

Theoretical Foundations of Generalization

Key Concepts:

- Generalization = performance on unseen tasks, not training ones
- Depends on task diversity, model capacity, and learning dynamics
- Evaluation happens on query set → across tasks, not within



Meta Overfitting and Gradient-Based Learning

Meta Overfitting:

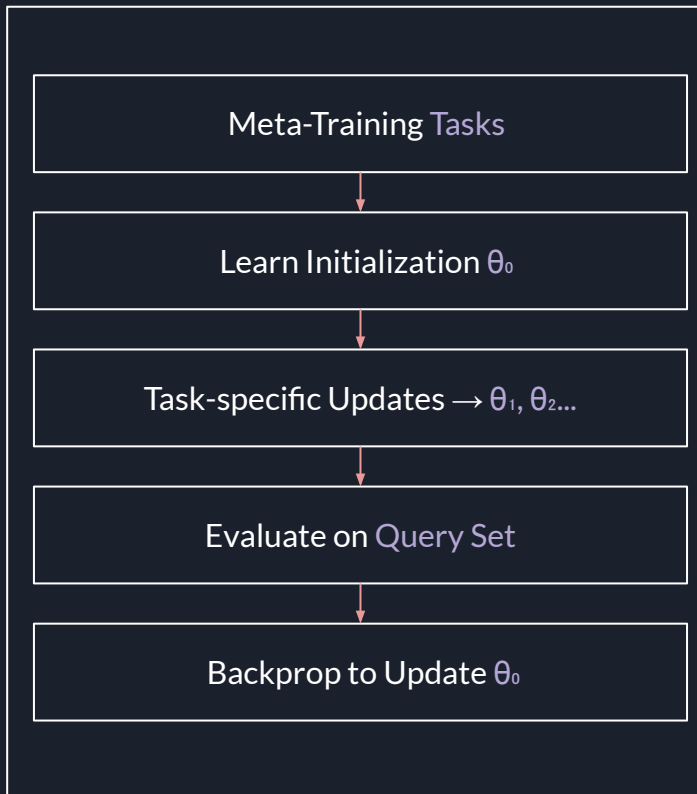
- Meta-learner overfits to training tasks → poor adaptation
- Caused by low task diversity or excessive inner-loop updates

Meta Regularization:

- Techniques like dropout, entropy regularization, task reweighting
- Reduce variance and improve robustness

Gradient Based Theory (eg. MAML):

- Treats meta-learning as bilevel optimization
- Outer loop: learns initialization
- Inner loop: adapts to task with few gradient steps.





Comparison between Meta-Learning and Transfer Learning

- **Meta Learning:** Meta learning is the process of training models to quickly adapt to new tasks by learning from a variety of related tasks.
- **Transfer Learning:** Transfer learning uses knowledge from one domain to improve learning in a different but related domain.

Aspect	Meta Learning	Transfer Learning
Data Requirement	Few-shot	Large dataset
Adaptation Speed	Fast	Slower (requires fine-tuning)
Task Awareness	Task-specific priors	Domain knowledge transfer
Generalization Target	Across tasks	Across domains



Week 5 Agenda: Probabilistic Meta Learning and Bayesian Perspectives

- Bayesian Meta Learning
- Uncertainty Modeling in few-shot learning
- Prior and posterior distributions across tasks
- Information theoretic frameworks for Meta Learning



What is Bayesian Meta-Learning

Bayesian Meta-Learning incorporates **probabilistic reasoning** into the meta-learning process

Core Idea:

- Model task uncertainty using distributions instead of deterministic parameter estimates.
- Encode knowledge across tasks using probabilistic priors and posteriors.
- Enables adaptive learning through posterior updates conditioned on task data.

Why It Matters:

- Captures epistemic and aleatoric uncertainties during few-shot adaptation.
- Enhances model robustness in data-scarce or noisy task environments.
- Promotes superior generalization across previously unseen meta-test distributions.

Example Methods:

- **Bayesian MAML**: Extends MAML by learning distributions over initializations.
- **Neural Processes**: Combines neural networks with stochastic process representations.
- **Bayes by Backprop** Performs variational inference to estimate weight posteriors.



Uncertainty in Few Shot Learning

Few-shot learning suffers from **limited supervision**, leading to high uncertainty

Traditional Methods:

- Use fixed weights without modeling any task-level uncertainty
- Perform poorly, under noisy, ambiguous, or limited training data
- Lack confidence estimation, making predictions hard to trust.

Bayesian Approaches:

- Model uncertainty by learning posterior distributions over parameters.
- Output predictive distributions instead of single-point estimates.
- Adapt to new tasks more reliably using uncertainty-aware updates.

Types of Uncertainty:

- **Epistemic:** Captures model uncertainty due to limited training data.
- **Aleatoric:** Represents inherent noise in observations or labels.
- Both types combined yield robust, uncertainty-aware meta-predictions.

Prior and Posterior across tasks

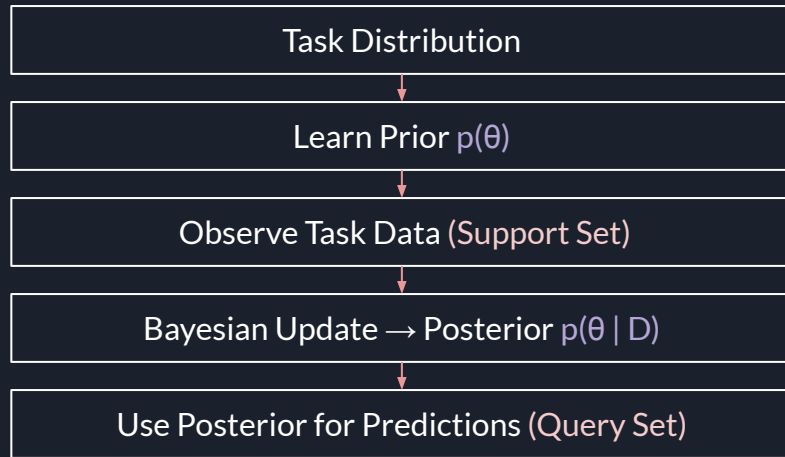
Prior is what the model believes before seeing data, and **posterior** is what it believes after updating with data.

- **Prior ($p(\theta)$)**: Belief about model parameters before seeing task-specific data. It encodes general knowledge across tasks.
- **Posterior ($p(\theta | D)$)**: Updated belief after observing task specific data. It is tailored to a particular task using Bayes' rule.

Bayesian Meta Learning flow: The model starts with a prior, uses few-shot data, and computes a posterior for task-specific adaptation

Why This Matters:

- Priors help initialize models for fast few-shot adaptation.
- Posteriors enable task-specific fine-tuning without overfitting.
- Supports uncertainty-aware and data-efficient learning across tasks.



Visualization – Bayesian Meta-Learning Flow

This flow illustrates how Bayesian meta-learning updates shared priors using task-specific data to make uncertainty-aware predictions and **improve generalization** across tasks.

Flow Summary:

- Sample multiple tasks from a distribution.
- Learn a shared prior across all tasks.
- Use each task's support set to update the posterior.
- Predict on the task's query set using the posterior.
- Evaluate performance and compute meta-objective.
- Update the prior to improve future task adaptation.

Task Distribution Sample multiple tasks ($T_1, T_2, \dots, T_n$)

Learn Shared Prior Meta-learner captures prior $p(\theta)$ across tasks

Observe Support Set Use task-specific support data D_{supp}

Posterior Update Update $p(\theta | D_{\text{u}})$ using Bayesian inference

Predict & Evaluate Use posterior to predict on D_{query}

Meta-Objective & Update Adjust prior using loss and gradient updates



Week 6 Agenda: Representation Learning for Meta and Few-Shot Learning

- Impact of learned representations on task transfer
- Unsupervised and self-supervised learning techniques
- Embedding space properties
- Relevance of Manifold learning theories in Meta Learning



Why Representation Learning Matters

Effect **representations** are key to fast, robust, and transferable learning.

Importance of Learned Representation:

- Help models capture essential patterns across tasks and domains
- Enable quick adaptation with minimal fine-tuning or supervision
- Crucial for handling task variability in meta-learning settings

Efficiency in Few-Shot Learning:

- Reduce sample complexity by reusing shared representations
- Boost accuracy with limited support examples per class
- Minimize overfitting in low-data task environments

Robustness and generalization:

- **Improve performance across noisy or shifted input distributions**
- **Strengthen resilience to label noise and data imbalance**
- **Support transfer to novel, unseen tasks or domains**



Key Techniques and Embedding Insights

Unsupervised Representation Learning:

- Learns patterns from unlabeled data without explicit supervision.
- Uses structures like autoencoders to compress input features.
- Clustering reveals inherent data groupings and task similarities.

Self-Supervised Techniques:

- Creates pseudo-labels from data to train useful embeddings.
- Contrastive methods (e.g., SimCLR) learn discriminative features.
- Boosts performance on downstream tasks with minimal labeling effort.

Embedding Space Characteristics:

- **Disentanglement** improves control over latent representations.
- **Clustering** helps organize tasks and supports few-shot transfer.
- **Separability** makes class boundaries more distinct and efficient.



Manifold Learning in Meta-Learning

Real-world data lies on **low-dimensional, non-linear manifolds**.

Manifold Hypothesis:

- Real-world data lies on structured, low-dimensional manifolds.
- High-dimensional data can often be compressed meaningfully.
- Manifolds reveal geometry critical for learning useful representations.

Role in Meta-Learning:

- Learns task manifolds to aid in few-shot adaptation.
- Better manifolds improve task interpolation and generalization.
- Embeddings aligned with manifolds enhance meta-task similarity modeling.

Tools and Outcomes:

- Methods like t-SNE and UMAP visualize high-dimensional structures.
- Spectral embeddings (e.g., ProtoNets) improve class separation.
- Stronger representations lead to faster, accurate few-shot learning.



THANK YOU!